Linux网络--Http请求与响应

- 1. 一次 HTTP 访问的完整路径
  - 1.1 URL 与 URI
  - 1.2 域名、DNS、IP 与 Host
  - 1.3 URL 编码
  - 1.4 HTTP 请求报文
  - 1.5 HTTP 响应报文
  - 1.6 序列化与反序列化
  - 1.7 正文边界与报文读取
    - 1.7.1 Content-Length
    - 1.7.2 Transfer-Encoding: chunked
    - 1.7.3 HTTP/2 与 HTTP/3
  - 1.8 常见 Header
  - 1.9 请求方法
  - 1.10 状态码
  - 1.11 重定向与 Location
  - 1.12 验证一次请求

1. 一次 HTTP 访问的完整路径

HTTP 是应用层协议，用来规定客户端和服务器如何交换资源或调用服务。一次浏览器访问可以抽象为：

```
用户输入 URL
  -> DNS 把域名解析为 IP
  -> 浏览器连接服务器的 IP:端口
  -> 发送 HTTP request
  -> 服务器读取并反序列化请求
  -> 处理静态资源或动态服务
  -> 返回 HTTP response
  -> 浏览器解释正文并继续请求子资源
```

HTTP/1.1 和 HTTP/2 通常运行在 TCP 上；HTTP/3 使用基于 UDP 的 QUIC。TCP 提供可靠、有序的字节流，但不保存 HTTP 消息边界，因此接收方必须依靠 HTTP 自身规则判断一条报文何时结束。

HTTP 本身是无状态协议，但“无状态”不等于“每次请求都重新建立连接”。HTTP/1.1 默认允许在同一个 TCP 连接上依次处理多个请求，浏览器还可能建立多条并行连接。

1.1 URL 与 URI

一个完整 URL 可以写成：

```
https://user:pass@www.example.jp:80/dir/index.htm?uid=1#ch1
```

部分	示例	含义
协议	`https`	应用层协议
认证信息	`user:pass@`	已不推荐在 URL 中传递凭据
主机名	`www.example.jp`	域名或 IP
端口	`80`	服务监听端口
路径	`/dir/index.htm`	资源或路由
查询字符串	`?uid=1`	GET 参数
片段	`#ch1`	文档内部位置，通常不发送给服务器

URI 是资源标识符，URL 是其中的定位形式。HTTP 请求行中的请求目标通常只保存路径和查询字符串：

```
GET /search?q=http HTTP/1.1
Host: example.com
```

完整 URL 是 `http://example.com/search?q=http`；请求目标只是 `/search?q=http`。

1.2 域名、DNS、IP 与 Host

人更容易记住域名，网络通信最终使用 IP 地址。浏览器通常通过系统递归 DNS 解析器查询 A 或 AAAA 记录：

```
www.example.com
  -> DNS 查询
  -> 93.184.216.34
  -> TCP 连接 93.184.216.34:443
```

域名不是 IP 地址本身，而是 IP 的可读映射。DNS 还涉及缓存、多级解析、负载均衡和 CDN 调度。

HTTP/1.1 请求必须携带 `Host`：

```
GET /index.html HTTP/1.1
Host: blog.example.com
```

多个域名可以解析到同一 IP。服务器根据 `Host` 区分不同网站，这称为虚拟主机。

1.3 URL 编码

空格、中文、`&`、`=`、`?`、`#`、`/` 等字符在 URL 中有特殊含义。参数值中出现这些字符时应编码：

```
C++ 教程
C%2B%2B%20%E6%95%99%E7%A8%8B
```

非 ASCII 字符先按 UTF-8 编码成字节，再逐字节写成 `%HH`。在 `application/x-www-form-urlencoded` 中，空格也可能编码成 `+`。

服务器应先按分隔符拆分参数，再分别解码键和值。若先整体解码，参数中原本编码过的 `&` 或 `=` 可能被误认为新的分隔符。

1.4 HTTP 请求报文

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
Connection: keep-alive

username=alice&password=123456
```

请求由四部分组成：

```
请求行：方法 SP 请求目标 SP HTTP 版本 CRLF
请求头：Key: Value CRLF
空行：CRLF
请求正文：可选
```

请求行有三个关键字段：方法、请求目标、HTTP 版本。Header 中保存目标主机、正文类型、长度、连接策略和 Cookie 等信息。

1.5 HTTP 响应报文

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 57
Connection: keep-alive

<!doctype html>
<html>
  <body>Hello, HTTP</body>
</html>
```

响应同样由四部分组成：

```
状态行：HTTP 版本 SP 状态码 SP 原因短语 CRLF
响应头：Key: Value CRLF
空行：CRLF
响应正文：可选
```

状态码是程序和自动化系统判断结果的主要依据，原因短语主要供人阅读，不应被客户端当作业务逻辑。

1.6 序列化与反序列化

序列化把内存结构转换为字节流，反序列化把字节流恢复为结构：

```
HttpRequest 对象
  -> 拼接请求行
  -> 拼接 Header
  -> 加入空行
  -> 拼接正文
  -> 字节流

字节流
  -> 查找 Header 结束空行
  -> 解析请求行
  -> 解析 Header
```

```
  -> 按长度读取正文
  -> HttpRequest 对象
```

HTTP/1.1 的 Header 字段名不区分大小写。`Set-Cookie` 可以出现多个实例，不能把所有同名 Header 简单合并成一个值。

1.7 正文边界与报文读取

接收方如果不知道正文在哪里结束，就会把下一条请求的一部分误认为当前正文，或者一直等待更多数据。

1.7.1 Content-Length

```
Content-Length: 57
```

它表示正文字节数，不是字符数。字符编码和 gzip 等内容编码都会影响实际发送长度。

1.7.2 Transfer-Encoding: chunked

当服务器无法提前知道正文长度时，可以使用分块传输：

```
Transfer-Encoding: chunked

5\r\n
hello\r\n
6\r\n
 world\r\n
0\r\n
\r\n
```

每个块先给十六进制长度，再给数据，最后用零长度块结束。HTTP/1.1 中 `Content-Length` 和 `Transfer-Encoding` 不应同时出现，冲突报文应被拒绝。

1.7.3 HTTP/2 与 HTTP/3

HTTP/2 使用 HEADERS 和 DATA 帧，HTTP/3 使用 QUIC 流，消息边界由二进制协议表达。

服务器至少需要：

1. 持续读取到 Header 结束的空行。
2. 限制请求行和 Header 的总大小。
3. 解析 `Content-Length` 或 `Transfer-Encoding`。
4. 循环读取到正文完整。
5. 防止长度溢出、超时和超大请求。
6. 循环发送响应，处理一次 `write()` 只写入部分数据的情况。

1.8 常见 Header

Header	方向	作用
`Host`	请求	目标主机和虚拟站点
`User-Agent`	请求	客户端信息，可被伪造
`Accept`	请求	可接受的响应类型
`Accept-Encoding`	请求	支持的压缩编码
`Accept-Language`	请求	偏好的语言
`Referer`	请求	来源页面
`Authorization`	请求	认证信息
`Cookie`	请求	发送 Cookie
`Content-Type`	请求/响应	正文媒体类型
`Content-Length`	请求/响应	正文字节数
`Connection`	请求/响应	连接是否复用
`Set-Cookie`	响应	设置 Cookie
`Location`	响应	重定向目标
`Cache-Control`	请求/响应	缓存策略
`ETag`	响应	资源版本
`Last-Modified`	响应	最后修改时间

`Content-Type` 说明正文是什么类型，`Content-Encoding` 说明内容是否使用 gzip、br 等压缩方式。

`User-Agent` 可用于兼容性判断，但不能作为安全身份。`Referer` 可用于访问来源统计和防盗链，但 URL 参数可能包含敏感信息。

1.9 请求方法

方法	语义	安全方法	幂等
GET	获取资源	是	是
HEAD	获取 Header，不返回正文	是	是
POST	提交数据或触发处理	否	通常否
PUT	替换目标资源	否	是
PATCH	部分修改资源	否	通常否
DELETE	删除资源	否	是
OPTIONS	查询选项或进行 CORS 预检	是	是
CONNECT	建立代理隧道	否	未作一般保证
TRACE	回显请求用于诊断	是	是

安全方法表示按规范不会修改服务器资源，不代表数据没有隐私风险。GET 参数可能出现在浏览器历史、日志和代理记录中。

HEAD 与 GET 的 Header 语义相似，但服务器不返回正文。OPTIONS 可以查询允许的方法，也用于跨域 CORS 预检。

1.10 状态码

范围	含义
1xx	信息性响应
2xx	成功
3xx	重定向或缓存验证
4xx	客户端请求问题
5xx	服务器处理问题

状态码	含义	常见场景
200	OK	请求成功
201	Created	资源创建成功
204	No Content	成功但没有正文
301	Moved Permanently	永久迁移
302	Found	临时重定向
303	See Other	要求改用 GET 获取结果
304	Not Modified	缓存未变化

状态码	含义	常见场景
307	Temporary Redirect	临时跳转并保持方法
308	Permanent Redirect	永久跳转并保持方法
400	Bad Request	请求格式或参数不合法
401	Unauthorized	未认证或认证失败
403	Forbidden	已识别身份但没有权限
404	Not Found	资源不存在
405	Method Not Allowed	路径存在但方法不支持
409	Conflict	与当前资源状态冲突
413	Content Too Large	正文过大
429	Too Many Requests	触发限流
500	Internal Server Error	服务器内部异常
502	Bad Gateway	网关从上游获得无效响应
503	Service Unavailable	服务暂不可用
504	Gateway Timeout	网关等待上游超时

401 与 403 的边界是“尚未认证”和“已认证但无权限”。100 Continue 通常与 `Expect: 100-continue` 配合，并不是 PUT 专用状态码。

1.11 重定向与 Location

```
HTTP/1.1 302 Found
Location: /dashboard
```

浏览器收到 3xx 后，会向 `Location` 地址重新发起请求。

状态码	永久性	方法语义
301	永久	客户端通常改为 GET
302	临时	历史行为通常改为 GET
303	临时	明确要求改为 GET
307	临时	保持原请求方法
308	永久	保持原请求方法

网站更换域名常用 301。搜索引擎可能据此把旧 URL 更新为新 URL，但浏览器书签是否自动修改由具体浏览器决定。

登录成功后的表单 POST 更适合使用 POST/Redirect/GET：服务器返回 303，让浏览器 GET 新页面，避免刷新时重复提交。

1.12 验证一次请求

```
curl -i http://127.0.0.1:8080/
curl -I http://127.0.0.1:8080/
curl -i -X OPTIONS http://127.0.0.1:8080/
```

也可以通过 telnet 手工发送：

```
GET / HTTP/1.1
Host: 127.0.0.1:8080

```

观察请求行、Host、响应状态行、Content-Type、Content-Length 和正文，可以把 URL、报文和状态码串成一条可验证的链路。

下一篇进入静态资源服务器：Linux网络--Http静态资源服务器